

Luphra MicroCover

Faruk Catak

June 6, 2026

Contents

1	Context and Scope	2
1.1	Problem	2
2	Goals and Non-Goals	2
2.1	Goals	2
2.2	Non-Goals	2
3	Design Overview	4
3.1	Insured Tool Sequence	4
3.2	Component Responsibilities	5
4	Detailed Design	5
4.1	Technical Stack	5
4.1.1	Platforms and Services	5
4.1.2	Python Packages	6
4.1.3	Tools Not Required	6
4.1.4	Research and Design Tools	6
4.1.5	Product Interface References	6
4.2	Interfaces	7
4.2.1	Codex Integration	7
4.2.2	HTTP API	7
4.2.3	Data Models	7
5	Alternatives Considered	8
6	Cross-Cutting Concerns	9
6.1	Security	9
6.2	Reliability and Error Handling	9
6.3	Observability	9
7	References	9

1 Context and Scope

Luphra MicroCover is a hackathon demonstration of per-action insurance for autonomous AI agents. An agent requests coverage before performing an economic action, receives a quote, pays a small premium through x402 on Algorand, and receives a coverage receipt.

The initial demonstration covers one scenario: a procurement agent buying API credits from a new vendor. It is intended to prove the payment and coverage workflow, not to provide a production insurance product.

1.1 Problem

AI agents can spend money, call tools, communicate with third parties, and make decisions on behalf of users. Existing controls generally allow or deny an action. They do not attach dynamically priced financial protection to an individual action.

Traditional insurance premiums are also poorly matched to agent activity. The risk can vary with the transaction amount, vendor, requested action, user policy, model confidence, and execution context.

2 Goals and Non-Goals

2.1 Goals

- Observe multiple Codex agents and stream their activity into one dashboard.
- Display an agent sidebar, central live activity stream, agent summary, and settlement panel.
- Evaluate a selected existing tool call before it executes.
- Allow harmless actions, deny unacceptable actions, and dynamically price coverage for allowed consequential actions.
- Collect the quoted premium automatically through x402 using Algorand TestNet USDC.
- Allow the original tool call only after coverage is confirmed.
- Return and display a coverage receipt containing the quote, coverage terms, and settlement reference.
- Record the tool outcome and publish it to the live dashboard.
- Keep the implementation small enough to explain and demonstrate live.

2.2 Non-Goals

- Production insurance underwriting, licensing, or claims administration.
- MainNet payments or real customer funds.
- Building a custom facilitator or operating an Algorand node.
- Deploying an AVM smart contract.
- Supporting EVM, Solana, subscriptions, or fiat checkout.

- Building custom business tools or an MCP proxy.
- Intercepting every Codex built-in tool or arbitrary network request.
- Building a production customer, policy-management, or claims dashboard.

3 Design Overview

3.1 Insured Tool Sequence

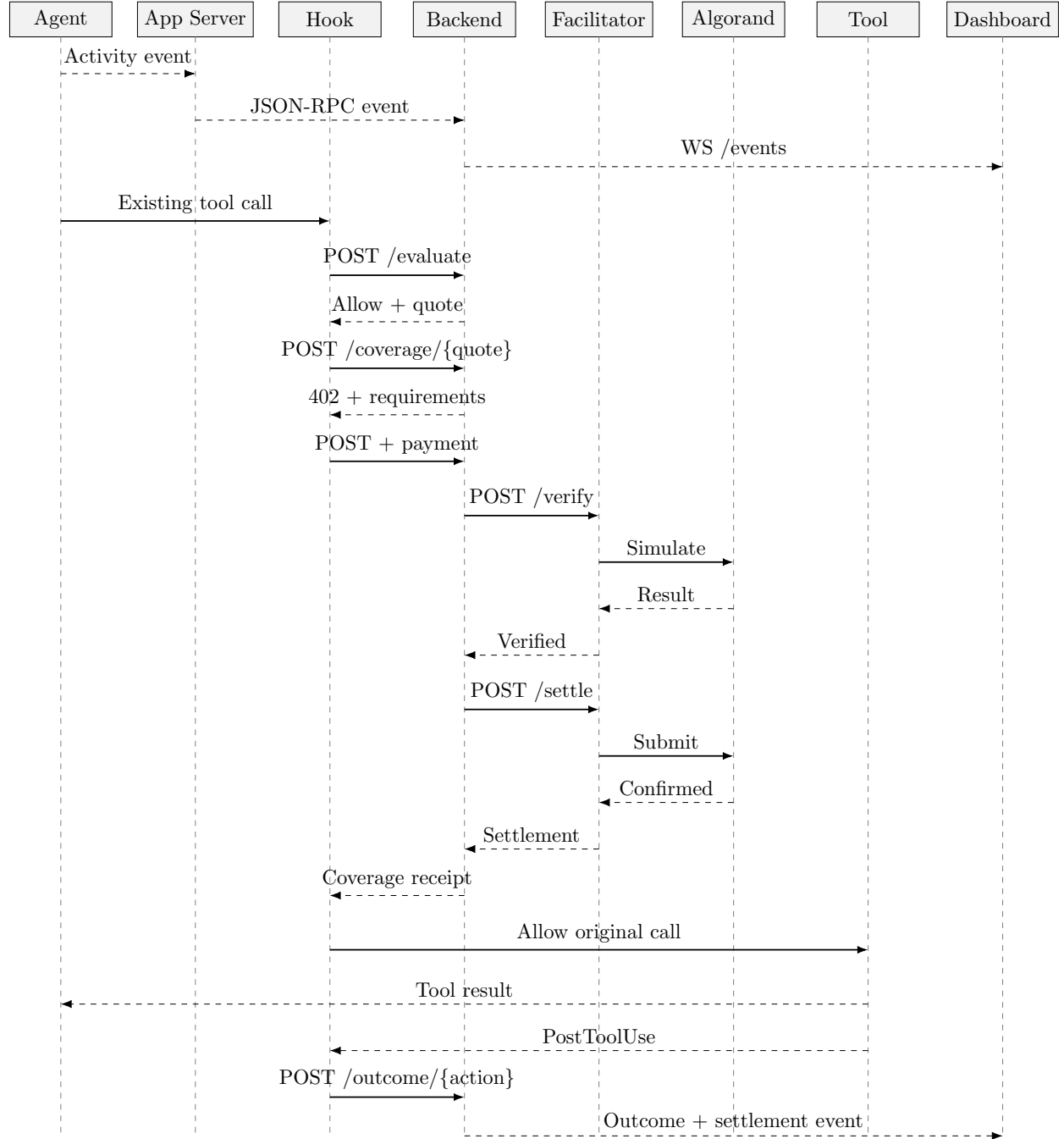


Figure 1: Continuous agent observation and hook-enforced coverage for an existing tool call.

3.2 Component Responsibilities

Component	Responsibility
Agent	Selects and calls an existing tool normally.
App Server	Emits continuous thread, turn, item, command, MCP, and status events. It provides observability but does not price risk or enforce coverage.
Hook	Receives <code>PreToolUse</code> , submits the pending tool name and arguments for coverage, pays the x402 premium, blocks on failure, and records the outcome through <code>PostToolUse</code> .
Backend	Prices the proposed action, hosts the x402-protected coverage resource, returns coverage receipts, combines App Server, hook, and settlement events, and publishes the unified dashboard stream.
Dashboard	Displays agents and continuous activity from App Server events alongside risk, coverage, and settlement details from the insurance path.
Facilitator	Validates the proposed payment, optionally supplies fee abstraction, submits the transaction group, and reports settlement.
Algorand	Records and finalizes the USDC payment.
Tool	Executes the original existing operation unchanged after the hook allows it.

4 Detailed Design

4.1 Technical Stack

4.1.1 Platforms and Services

Technology	Use
Python 3.12	Application language.
FastAPI	Risk and coverage API, x402 resource server, and dashboard event relay.
Codex Hooks	Runs synchronous <code>PreToolUse</code> and <code>PostToolUse</code> scripts around selected tool calls.
Codex App Server	Supplies the continuous thread, turn, item, command, MCP, and status event stream used by the agent sidebar and live activity view. It is not part of insurance enforcement.
Existing tool or MCP server	Performs the demonstrated action without source changes or a custom wrapper.
GoPlausible facilitator	Hosted verification and settlement service.
Algorand TestNet USDC	Demonstration payment asset.
LORA	Transaction inspection and debugging.

4.1.2 Python Packages

Package	Reason
x402-avm[fastapi,avm,httpx]	Provides the FastAPI payment middleware, Algorand payment mechanism, and automatic paying client integration.
uvicorn	Runs the FastAPI application and supplies standard WebSocket support.
websockets	Connects the Backend to Codex App Server. It is installed by <code>uvicorn[standard]</code> .
openai	Calls a small model with structured output for the allow-or-deny decision and dynamic risk assessment.
pydantic	Defines the structured underwriting output and validates API and dashboard models. It is declared directly because application code imports it.
python-dotenv	Loads local configuration from the uncommitted <code>.env</code> file.
pytest	Runs backend and payment-flow tests.
ruff	Formats and lints the Python source.
pyright	Performs static type checking.

FastAPI, HTTPX, and the Algorand SDK are installed transitively by the selected `x402-avm` extras and are not repeated as direct project dependencies.

4.1.3 Tools Not Required

AlgoKit, NodeKit, a local Algorand node, and AVM smart-contract tooling are not required for the initial design. They become relevant only if the project later introduces smart contracts, LocalNet development, or self-hosted infrastructure.

4.1.4 Research and Design Tools

Tool	Use
Tavily CLI	Searches and extracts current technical documentation, product information, and supporting sources during research.
Mobbin	Provides references for established application flows and interface patterns.
Refero MCP	Lets the development agent search real product screens, flows, and dashboard patterns while designing the demonstration interface.

4.1.5 Product Interface References

Product	Relevant Pattern
Conductor	Provides a reference for a desktop control panel that shows multiple agents, their activity, and items requiring attention.
Clawvisor	Provides a reference for presenting agent risk, authorization decisions, policy enforcement, and an action audit trail.
Allowance	Provides a reference for transaction approval, spending constraints, and scoped payment credentials for agent purchases.

4.2 Interfaces

4.2.1 Codex Integration

Integration	Purpose	Interfaces Used
App Server	Observes agents and supplies live activity for the sidebar and activity view.	JSON-RPC input; WS <code>/events</code> output.
Hooks	Allows or blocks selected tool calls and records their results.	POST <code>/evaluate</code> , POST <code>/coverage/{quote_id}</code> , and POST <code>/outcome/{action_id}</code> .

App Server provides observability; hooks provide insurance enforcement.

4.2.2 HTTP API

Endpoint	Purpose
POST <code>/evaluate</code>	Receives a proposed tool call. It first allows or denies the action and, only when allowed, returns any required risk assessment and quote.
POST <code>/coverage/{quote_id}</code>	x402-protected endpoint that activates the quoted coverage after payment.
POST <code>/outcome/{action_id}</code>	Records the existing tool's success, failure, or cancellation.
WS <code>/events</code>	Sends an initial dashboard snapshot and then a unified stream of App Server activity, hook decisions, risk, coverage, settlement, and outcome events.

The backend keeps the current agents, actions, quotes, receipts, outcomes, and aggregate statistics in memory. When the dashboard connects, the first WebSocket message is a complete snapshot. The sidebar and all dashboard panels can therefore render without separate agent or statistics routes. Subsequent messages contain only incremental updates.

4.2.3 Data Models

FastAPI's existing Pydantic integration will define and validate six compact models:

- `AgentState`
- `ToolAction`
- `Evaluation`, containing the policy decision, risk assessment, and optional quote.
- `CoverageReceipt`
- `ToolOutcome`
- `DashboardEvent`, including the initial snapshot variant.

The agent is created or refreshed automatically from the identifier and display name included with each evaluated tool call. No agent registration or lookup endpoint is required for the demonstration.

5 Alternatives Considered

Decision	Alternatives	Selected Approach
Agent integration	Instructions only; custom MCP tools; MCP proxy; Codex hooks.	Use hooks around selected existing tools, including MCP tools where useful. Instructions cannot enforce coverage, while custom tools and a proxy add unnecessary implementation work.
Agent monitoring	Hook events only; CLI JSONL output; polling; Codex App Server.	Use App Server events because hooks expose only insured calls, while the dashboard needs continuous multi-agent activity.
Insurance timing	Observe actions after execution; require a separate manual coverage step; intercept before execution.	Use <code>PreToolUse</code> so payment and coverage complete before the existing tool runs, followed by <code>PostToolUse</code> for its outcome.
Risk decision	Fixed rules; unrestricted model decision; structured model output.	Use a small model with validated structured output for allow, deny, risk, and price. The result remains explicit and displayable.
Pricing	Fixed premium; subscription; dynamic per-action premium.	Use a dynamic premium bound to a server-side quote identifier. This directly demonstrates Lumphra’s per-action insurance model.
Payment asset	Native ALGO; TestNet USDC.	Use TestNet USDC so premiums have a stable, understandable value.
Settlement	Smart contract; ordinary asset transfer through a facilitator.	Use the facilitator and ordinary transfer. A smart contract does not improve the core hackathon demonstration.
Infrastructure	Self-hosted facilitator and Algorand node; hosted facilitator and public TestNet access.	Use hosted infrastructure to keep the project focused on agent insurance.
Dashboard transport	Separate polling routes for agents, statistics, actions, and settlements; one WebSocket with a snapshot and incremental events.	Use <code>WS /events</code> . The initial snapshot renders the complete dashboard; later messages update it without extra read routes.
Dashboard scope	Chat interface; insurer analytics dashboard; agent control room.	Use an observational control room: agent list and activity in the center, agent summary at the top right, and coverage settlements at the bottom right.

6 Cross-Cutting Concerns

6.1 Security

- Never commit private keys or the local `.env` file.
- Use separate payer and receiver TestNet accounts.
- Validate that a quote cannot be reused or paid with an altered amount.
- Return coverage only after successful verification and settlement.
- Keep the TestNet payer key outside hook logs and tool arguments.
- Fail closed when the hook cannot obtain a valid coverage receipt.
- Treat all facilitator and client payloads as untrusted input.

6.2 Reliability and Error Handling

The system must distinguish an ordinary payment-required response from failed pricing, verification, or settlement. The hook performs at most one automatic payment-bearing retry and blocks the pending tool call if coverage is not confirmed. Duplicate hook invocations and ambiguous settlement outcomes require idempotency before the design can move beyond a demo.

6.3 Observability

The middle dashboard panel displays normalized hook and tool events. The settlement panel displays the tool name, quote identifier, payer address, premium, coverage limit, settlement transaction ID, and a LORA link where available. Private keys, raw reasoning, complete tool arguments, and signed payloads must never be logged or sent to the browser.

7 References

- [1] 0xMihej, AI Agents Berlin, and Algorand Foundation, “Algorand Builders Berlin: Agentic Commerce x402 Hackathon \$21,000+ Prize-Pool,” *Luma*, June 6–7, 2026. Available: <https://lu.ma/agentic-commerce-hack>. Accessed: June 5, 2026.
- [2] Algorand Foundation, “x402 on Algorand,” *Algorand Developer Portal*, n.d. Available: <https://dev.algorand.co/resources/x402-on-algorand/>. Accessed: June 5, 2026.
- [3] GoPlausible, “x402-avm Python FastAPI Middleware Examples,” *GoPlausible x402-avm Documentation*, n.d. Available: <https://github.com/GoPlausible/.github/blob/main/profile/algorand-x402-documentation/python/x402-avm-fastapi-examples-python.md>. Accessed: June 5, 2026.
- [4] GoPlausible, “x402-avm V2 AVM Mechanism Examples (Python),” *GoPlausible x402-avm Documentation*, n.d. Available: <https://github.com/GoPlausible/.github/blob/main/profile/algorand-x402-documentation/python/x402-avm-avm-examples-python.md>. Accessed: June 5, 2026.

- [5] GoPlausible, “x402-avm Python httpx Client Examples,” *GoPlausible x402-avm Documentation*, n.d. Available: <https://github.com/GoPlausible/.github/blob/main/profile/algorand-x402-documentation/python/x402-avm-httpx-examples-python.md>. Accessed: June 5, 2026.
- [6] Algorand Foundation, “Lora Overview,” *Algorand Developer Portal*, n.d. Available: <https://dev.algorand.co/algokit/lora/overview>. Accessed: June 6, 2026.
- [7] Melty Labs, “Conductor: Run a Team of Coding Agents on Your Mac,” *Conductor*, n.d. Available: <https://conductor.build>. Accessed: June 6, 2026.
- [8] Clawvisor, “Clawvisor: The Authorization Layer for AI Agents,” *Y Combinator*, 2026. Available: <https://www.ycombinator.com/companies/clawvisor>. Accessed: June 6, 2026.
- [9] Allowance, “Allowance: Scoped Payment Credentials for AI Agents,” *Y Combinator*, 2026. Available: <https://www.ycombinator.com/companies/allowance>. Accessed: June 6, 2026.
- [10] OpenAI, “Codex App Server,” *OpenAI Developers*, n.d. Available: <https://developers.openai.com/codex/app-server>. Accessed: June 6, 2026.
- [11] OpenAI, “Hooks,” *OpenAI Developers*, n.d. Available: <https://developers.openai.com/codex/hooks>. Accessed: June 6, 2026.